

Instituto Tecnológico Superior de Monclova

Ingeniería en Informática

Unidad 2: Diseño de bases de datos con el modelo E-R

Tarea 2.2

Transición de diagrama E-R a notación UML

Comparativa E-R vs. UML: taller automotriz

Materia: Fundamentos de Base de Datos

Alumno(a): Kevin Anselmo Orozco Aguayo

Matrícula: I24050540

Docente: Rubén Riojas

Fecha de entrega: 03 de noviembre de 2026

Índice

1. Fundamentos **3**

1.1 Modelo entidad-relación 3

1.2 Diagrama de clases UML 3

2. Reglas de conversión de E-R a UML **3**

3. Aplicación al caso del taller automotriz **4**

3.1 Entidades y atributos 4

3.2 Relaciones 5

3.3 Decisiones de conversión 6

4. Comparativa detallada E-R vs. UML **11**

5. Análisis de coincidencias para el desarrollo orientado a objetos **11**

5.1 Coincidencias 11

5.2 Diferencias a considerar 12

5.3 Implicaciones para la implementación 12

6. Conclusiones **13**

Introducción

El diagrama entidad-relación describe qué datos se almacenarán y cómo se relacionan; el diagrama de clases de UML describe las clases de un sistema orientado a objetos, sus atributos, operaciones y asociaciones. Como ambos modelos parten de los mismos conceptos del problema, es posible convertir uno en el otro de manera casi directa. Esta correspondencia es útil porque permite que el diseño de la base de datos y el diseño del software mantengan la misma estructura.

En este documento se toma el diagrama E-R/E-RE del taller automotriz (Tarea 2.1) y se convierte en un diagrama de clases UML. Se establecen las reglas de conversión, se aplican al caso, se presenta una comparativa detallada entre ambos modelos y se analizan las coincidencias y diferencias para el desarrollo orientado a objetos.

1. Fundamentos

1.1 Modelo entidad-relación

El modelo E-R representa la información mediante entidades, atributos y relaciones (Chen, 1976). Su versión extendida añade especialización, generalización y agregación (Elmasri & Navathe, 2016). Es un modelo conceptual: no depende de un sistema gestor ni de un lenguaje de programación.

1.2 Diagrama de clases UML

El lenguaje unificado de modelado (UML) es un estándar para visualizar, especificar y documentar sistemas de software (Booch et al., 2005; Object Management Group, 2017). El diagrama de clases muestra las clases del sistema, cada una con un nombre, atributos y operaciones, y las relaciones entre ellas: asociaciones con multiplicidad, agregación, composición y generalización (Fowler, 2004).

2. Reglas de conversión de E-R a UML

La Tabla 1 resume las reglas que se aplicaron para convertir cada elemento del modelo E-R en su equivalente de UML.

Tabla 1

Reglas de conversión de E-R a UML

Elemento del modelo E-R	Elemento en UML	Regla de conversión
Entidad	Clase	Cada entidad se convierte en una clase con el mismo nombre (en singular).

Elemento del modelo E-R	Elemento en UML	Regla de conversión
Atributo simple	Atributo de la clase	Se conserva con su tipo de dato y visibilidad privada (-).
Atributo clave	Atributo identificador	UML no tiene claves; se conserva como atributo (idCliente, folio) para identificar los objetos.
Atributo compuesto	Atributos separados o clase de tipo de dato	Se descompone en sus partes (nombre y apellidos).
Atributo derivado	Atributo derivado (/)	Se marca con una diagonal; se calcula mediante una operación.
Atributo multivalorado	Atributo con multiplicidad	Se expresa con una colección: String[1..*] o String[0..*].
Relación binaria	Asociación	Se traza una línea entre las clases con el nombre de la relación.
Cardinalidad (1:1, 1:N, M:N)	Multiplicidad	1:N se expresa como 1 y 0..* o 1..*; M:N como 0..* en ambos extremos.
Participación total o parcial	Multiplicidad mínima 1 o 0	Total equivale a un mínimo de 1; parcial, a un mínimo de 0.
Atributo de relación	Clase de asociación	La relación con atributos se vuelve una clase unida a la asociación con línea punteada.
Entidad débil y relación identificadora	Composición	La clase dependiente se une con rombo relleno a la clase propietaria.
Especialización / generalización	Generalización (herencia)	Las subclases apuntan con triángulo hueco a la superclase; se anota {disjoint, complete}.
Agregación E-R (relación como entidad)	Clase para la relación agregada	La relación INCLUYE se vuelve la clase DetalleServicio, que se asocia con otras clases.

3. Aplicación al caso del taller automotriz

3.1 Entidades y atributos

La Tabla 2 muestra cómo cada entidad del diagrama E-R/E-RE se convirtió en una clase, incluidos los atributos especiales.

Tabla 2

Correspondencia de entidades y atributos con clases

Entidad E-R	Clase UML	Observaciones
CLIENTE	Cliente	Atributos nombre y apellidos (compuesto) y telefonos[1..*] (multivalorado).
VEHÍCULO	Vehiculo	/antiguedad derivada del atributo anio.
ORDEN_SERVICIO	OrdenServicio	/total derivado, calculado con calcularTotal().
SERVICIO	Servicio	Catálogo de servicios; se relaciona mediante DetalleServicio.
REFACCIÓN	Refaccion	Operación descontarExistencia() para actualizar el inventario.

Entidad E-R	Clase UML	Observaciones
PROVEEDOR	Proveedor	Sin cambios respecto al modelo E-R.
PAGO (entidad débil)	Pago	Se une a OrdenServicio mediante composición.
EMPLEADO	Empleado (abstracta)	Superclase; no se instancia directamente porque todo empleado es de un tipo.
RECEPCIONISTA, MECÁNICO, ALMACENISTA	Recepcionista, Mecanico, Almacenista	Subclases que heredan de Empleado; certificaciones[0..*] es multivalorado.
Agregación INCLUYE	DetalleServicio	Clase que representa un servicio dentro de una orden.
Atributos de EJECUTA y USA	Ejecucion y UsoRefaccion	Clases de asociación con horasTrabajadas y cantidad.

3.2 Relaciones

Las relaciones del modelo E-R se transformaron en asociaciones, composiciones, agregaciones y generalizaciones, con las multiplicidades que resultan de la cardinalidad y la participación (Tabla 3). Por ejemplo, la participación total de ORDEN_SERVICIO en INGRESA y la cardinalidad 1:N producen la multiplicidad 1 del lado del vehículo y 0..* del lado de la orden.

Tabla 3

Correspondencia de relaciones con asociaciones

Relación E-R	Asociación UML	Multiplicidad	Tipo
POSEE (CLIENTE–VEHÍCULO)	Cliente – Vehiculo	1 a 1..*	Agregación
INGRESA (VEHÍCULO–ORDEN)	Vehiculo – OrdenServicio	1 a 0..*	Asociación
REGISTRA (RECEPCIONISTA–ORDEN)	Recepcionista – OrdenServicio	1 a 0..*	Asociación
INCLUYE (ORDEN–SERVICIO)	OrdenServicio – DetalleServicio – Servicio	1 a 1..* y 0..* a 1	Composición y asociación
EJECUTA (MECÁNICO–INCLUYE)	DetalleServicio – Mecanico	0..* a 1..*	Asociación con clase Ejecucion
USA (INCLUYE–REFACCIÓN)	DetalleServicio – Refaccion	0..* a 0..*	Asociación con clase UsoRefaccion
SUMINISTRA (PROVEEDOR–REFACCIÓN)	Proveedor – Refaccion	1 a 0..*	Asociación
GENERA (ORDEN–PAGO)	OrdenServicio – Pago	1 a 0..*	Composición
Especialización de EMPLEADO	Recepcionista, Mecanico, Almacenista hacia Empleado	{disjoint, complete}	Generalización

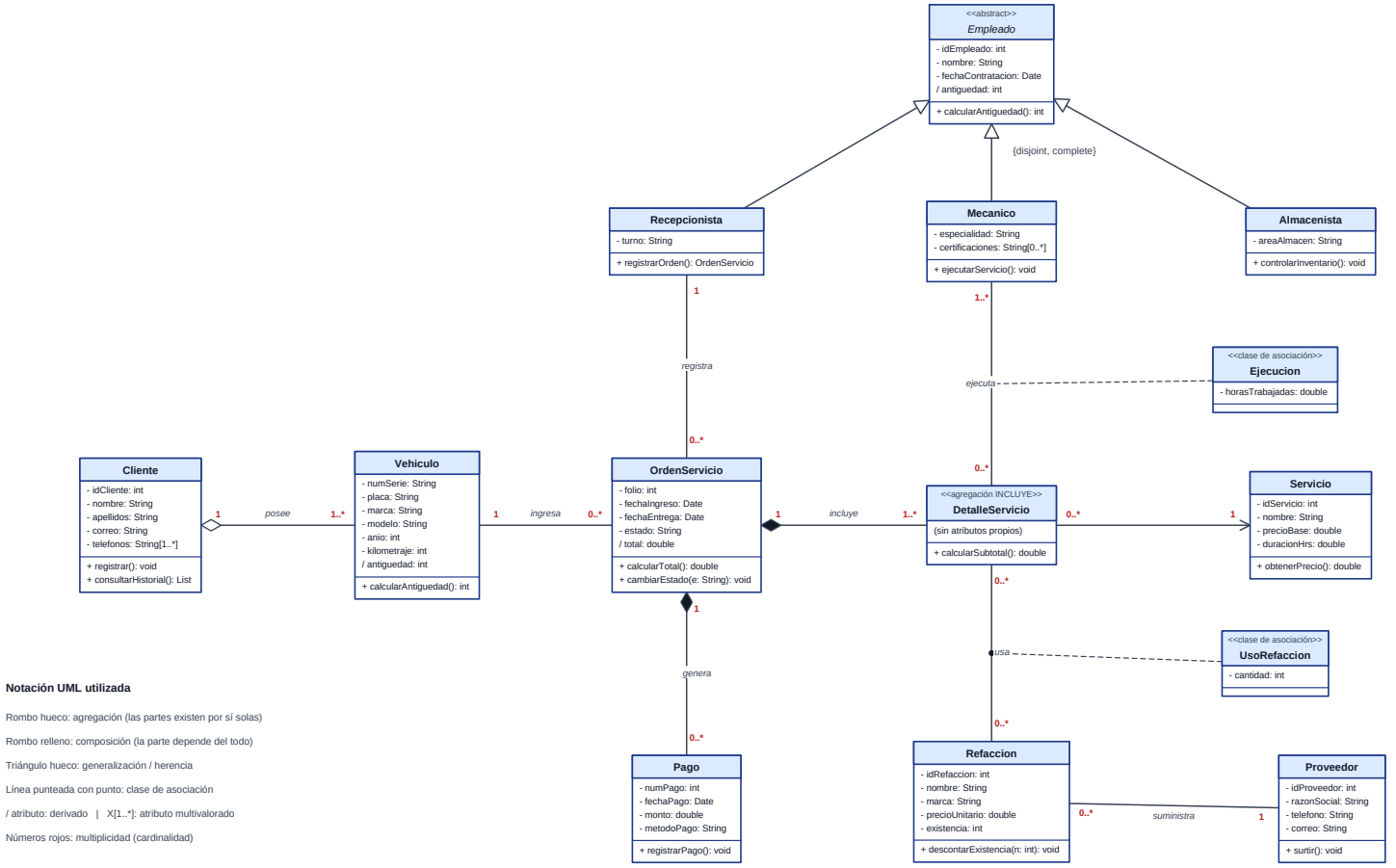
3.3 Decisiones de conversión

- **Composición para PAGO.** Como entidad débil, un pago no existe sin su orden; en UML se modela con composición, y los pagos desaparecen si se elimina la orden.
- **DetalleServicio.** La agregación INCLUYE es una relación que se relaciona con otras entidades. En UML se representa con una clase propia, compuesta por la orden y asociada con el servicio, los mecánicos y las refacciones.
- **Clases de asociación.** Los atributos horas_trabajadas y cantidad pertenecen al vínculo y no a una sola clase; por eso se convierten en las clases Ejecucion y UsoRefaccion.
- **Superclase abstracta.** Empleado se marca como abstracta, porque la especialización es total: no existen empleados que no sean recepcionistas, mecánicos o almacenistas.
- **Operaciones.** A diferencia del modelo E-R, UML permite incluir operaciones; se añadieron las mínimas que dan sentido a los atributos derivados y a las reglas de negocio (por ejemplo, calcularTotal).

La Figura 1 muestra el diagrama de clases completo y las Figuras 2 a 4 presentan acercamientos. El archivo fuente editable (.drawio) y las exportaciones en PNG, SVG y PDF se encuentran en el repositorio.

Figura 1

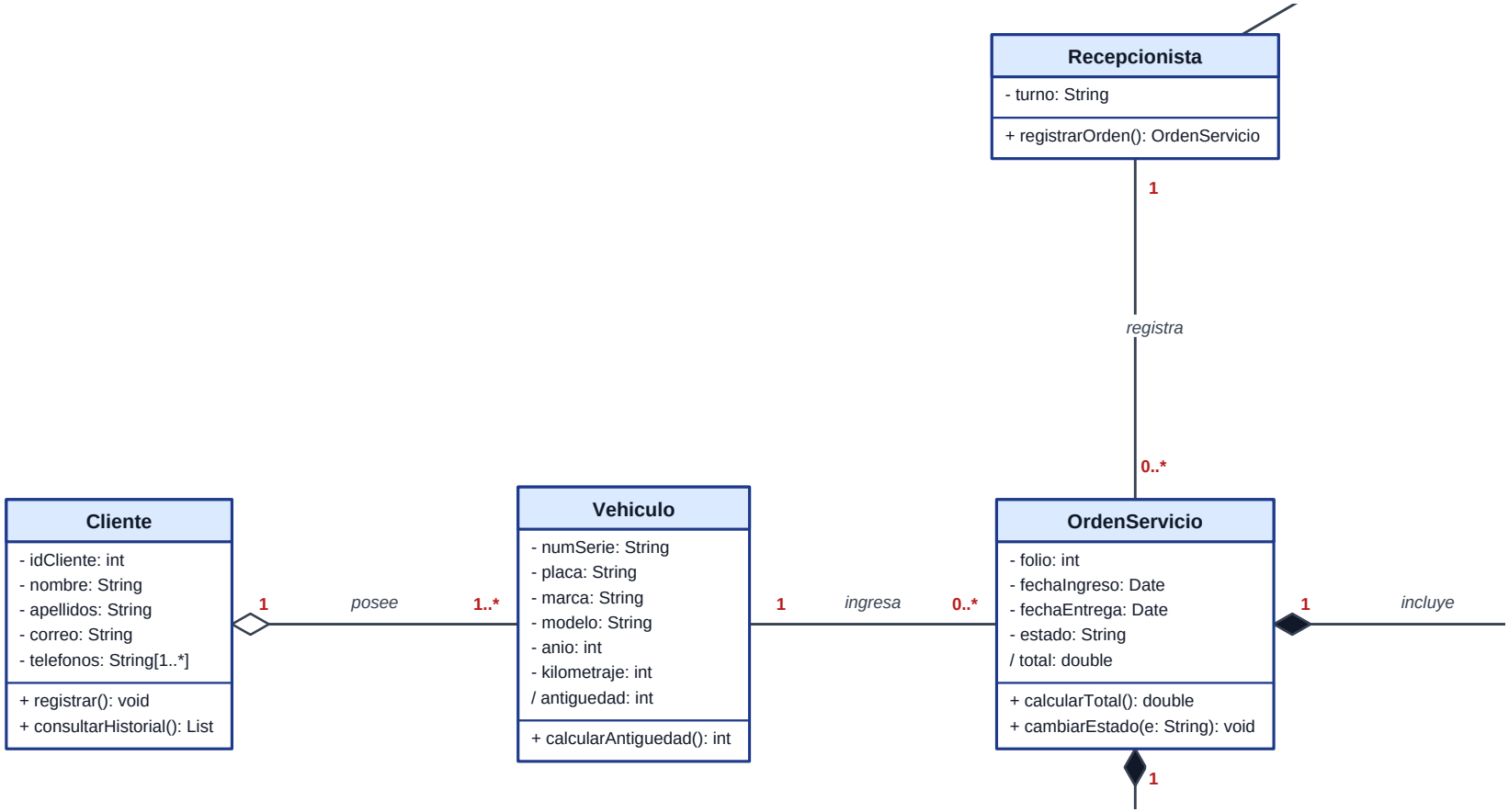
Diagrama de clases UML completo del taller automotriz



Nota. Elaboración propia con draw.io a partir del diagrama E-R/E-RE de la Tarea 2.1.

Figura 2

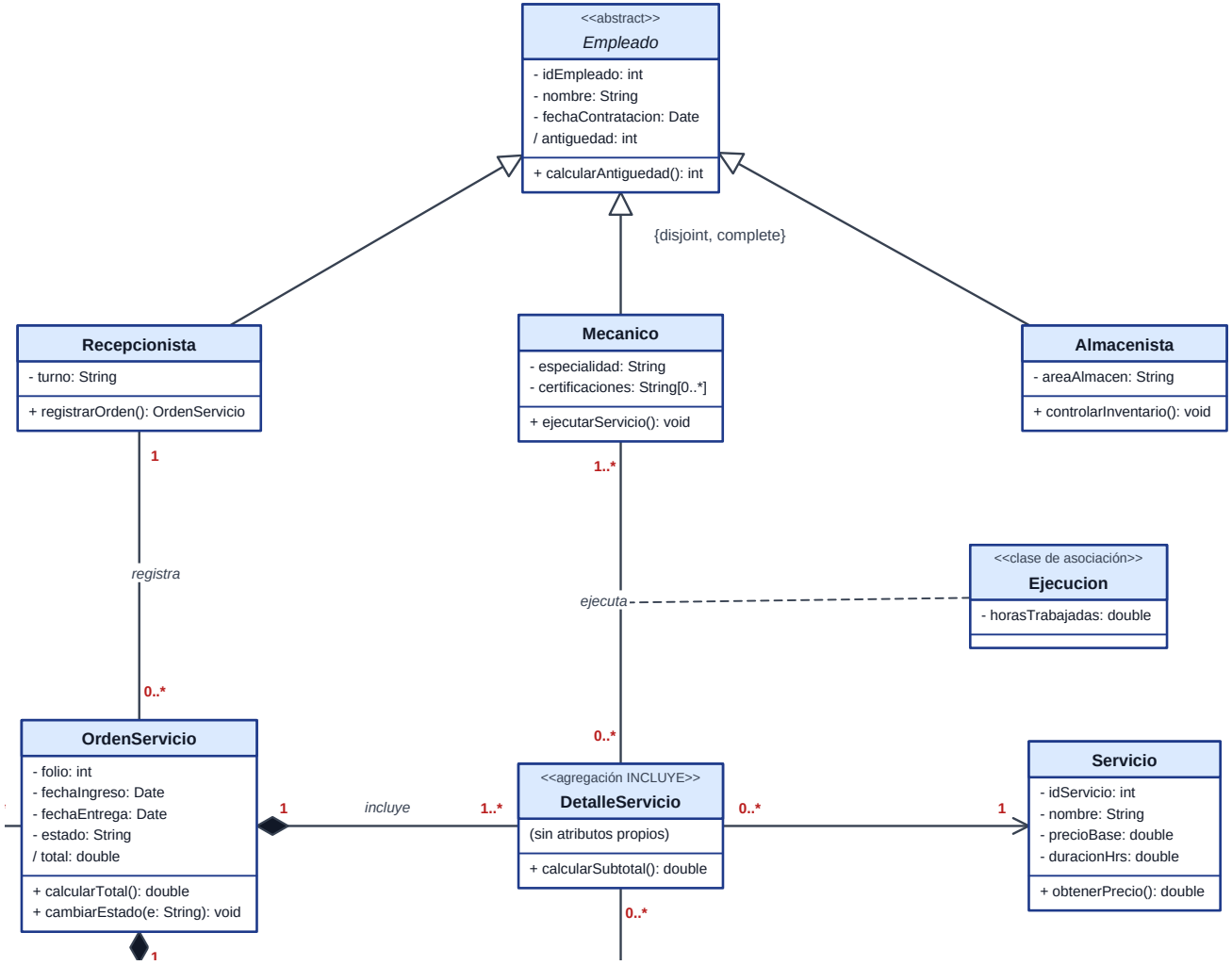
Acercamiento 1: clientes, vehículos, órdenes y recepcionistas



Nota. Elaboración propia con draw.io. Rombo hueco: agregación.

Figura 3

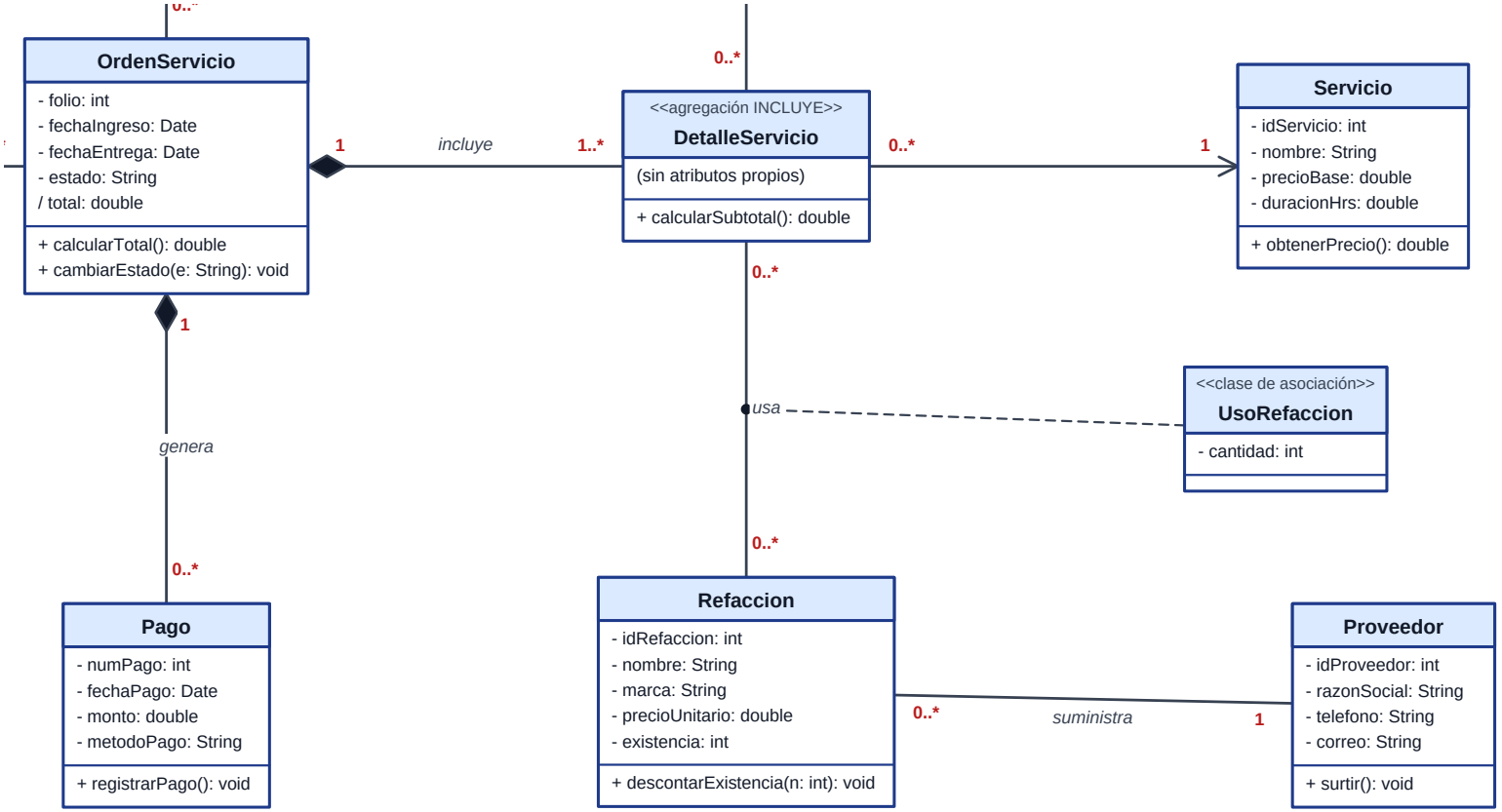
Acercamiento 2: herencia de empleados y ejecución de servicios



Nota. Elaboración propia con draw.io. Triángulo hueco: generalización; línea punteada: clase de asociación.

Figura 4

Acercamiento 3: detalle de servicio, refacciones, proveedores y pagos



Nota. Elaboración propia con draw.io. Rombo relleno: composición.

4. Comparativa detallada E-R vs. UML

Tabla 4

Comparativa detallada entre el modelo E-R y el diagrama de clases UML

Aspecto	Modelo E-R / E-RE	Diagrama de clases UML
Propósito	Diseñar la estructura de los datos que se almacenarán.	Modelar la estructura y el comportamiento de los objetos de un sistema de software.
Elemento principal	Entidad: conjunto de ocurrencias que se almacenan.	Clase: plantilla de objetos con atributos y operaciones.
Comportamiento	No se modela; solo datos y restricciones.	Se modela mediante operaciones (métodos) en cada clase.
Atributos	Clave, compuesto, derivado y multivalorado con símbolos propios.	Atributos con tipo y visibilidad; derivado con (/); multivalorado con multiplicidad.
Identificación	La clave es un concepto central.	No hay claves; los objetos tienen identidad propia (se conserva el atributo identificador).
Relaciones	Rombo entre entidades, con cardinalidad (1, N, M).	Asociación con multiplicidad (0..1, 1, 0..*, 1..*) y, opcionalmente, nombre, roles y navegabilidad.
Participación	Total (línea doble) o parcial.	Mínimo 1 o 0 en la multiplicidad.
Atributos de relación	Atributos colgados del rombo.	Clase de asociación.
Entidad débil	Rectángulo doble con relación identificadora.	Composición (rombo relleno).
Especialización	Círculo con d u o, línea doble si es total.	Generalización con triángulo hueco y restricciones {disjoint, complete}.
Agregación	Relación tratada como entidad (caja punteada).	Clase para la relación, o agregación (rombo hueco) entre partes y todo.
Nivel de abstracción	Conceptual: independiente del SGBD.	Conceptual, de diseño o de implementación, según el detalle.
Resultado posterior	Esquema relacional (tablas y claves foráneas).	Código orientado a objetos (clases, atributos, métodos).

5. Análisis de coincidencias para el desarrollo orientado a objetos

5.1 Coincidencias

- Las entidades y las clases representan los mismos conceptos del problema (cliente, vehículo, orden), por lo que su identificación coincide casi siempre.
- Los atributos se corresponden uno a uno; los derivados y multivalorados tienen notación equivalente en ambos modelos.
- La cardinalidad del E-R equivale a la multiplicidad de UML, y la participación total o parcial equivale a un mínimo de 1 o de 0.

- La especialización del E-R es la herencia de UML, con las mismas restricciones de disjunción y completitud.
- La entidad débil corresponde a la composición, pues en ambos casos la parte depende del todo.

5.2 Diferencias a considerar

- **Claves.** En la base de datos cada fila se identifica por una clave; en los objetos la identidad es intrínseca y las referencias se guardan como objetos, no como claves foráneas.
- **Comportamiento.** Las clases incluyen operaciones y el modelo E-R no; la lógica de negocio debe agregarse en el diseño de clases.
- **Relaciones M:N.** En el modelo relacional se resuelven con una tabla intermedia; en las clases se pueden representar con colecciones o con una clase de asociación, como en DetalleServicio.
- **Navegabilidad.** UML indica en qué dirección se recorre una asociación (por ejemplo, de DetalleServicio hacia Servicio); el E-R no tiene este concepto.

5.3 Implicaciones para la implementación

Al programar, cada clase del diagrama se traduce en una clase del lenguaje elegido y cada asociación en una referencia o colección. A modo de ejemplo, el fragmento siguiente muestra cómo se verían Cliente y OrdenServicio:

```
class Cliente {
    private int idCliente;
    private String nombre;
    private String apellidos;
    private String correo;
    private List<String> telefonos;        // atributo multivalorado [1..*]
    private List<Vehiculo> vehiculos;      // asociación Cliente 1 - 1..* Vehiculo
}

class OrdenServicio {
    private int folio;
    private Vehiculo vehiculo;             // asociación INGRESA
    private List<DetalleServicio> detalles;// composición: sin orden no existen
    private List<Pago> pagos;              // composición: entidad débil
    public double calcularTotal() { ... }  // atributo derivado /total
}
```

Al guardar estos objetos en una base de datos relacional se utiliza un mapeo objeto-relacional: normalmente cada clase se convierte en una tabla, las asociaciones en claves foráneas, los atributos multivalorados en tablas aparte y la herencia se resuelve con una de varias estrategias (una tabla por jerarquía o una por clase). Haber partido de un diseño

E-R/E-RE previo facilita ese paso, porque la estructura de datos y la de clases ya son coherentes (Elmasri & Navathe, 2016).

6. Conclusiones

La conversión del diagrama E-R/E-RE a UML fue directa en la mayoría de los elementos: entidades a clases, atributos a atributos, relaciones a asociaciones, cardinalidades a multiplicidades y especialización a herencia. Los elementos con más decisiones de diseño fueron la agregación, que se resolvió con la clase DetalleServicio, y los atributos de relación, que se convirtieron en clases de asociación.

La principal aportación de UML respecto al E-R es el comportamiento: las operaciones de cada clase. A cambio, el E-R es más preciso para describir claves y restricciones de los datos. Usar ambos modelos de forma complementaria permite que la base de datos y la aplicación se diseñen con la misma estructura y se mantengan alineadas durante el desarrollo.

Referencias

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The unified modeling language user guide* (2.ª ed.). Addison-Wesley.

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9–36. https://doi.org/10.1145/320434.320440

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML)* (Versión 2.5.1). https://www.omg.org/spec/UML/2.5.1